

Ziel

> Ich möchte, anhand der Typregeln ein paar nice Graphen zeigen > Ich möchte die formalen Properties aufzeigen > Ich möchte den Algorithmus zeigen

Pro Slide

- Nicht zu sehr auf die Einzelheiten (die dort stehen)
- Sondern mehr auf die Intuition und slides as “reference”
- An sich aber eigentlich: auf slides nur wichtiges ↴
- Dafür vielleicht den Rest einfach ausgrauen?
- Oder doch einfach massiv viele Slides?

Proofs and Definitions

2.1 Transparency of monitors p.5 (sound & complete) 2.2 Preciseness of deadlock detection p.5 (sound & complete) 3.6 Lock p.9 3.7 Deadlock p.9 4.4 Monitor instrumentation p.12 4.5 Monitor de-instrumentation p.12 4.6 Prop: $M^{(-1)}(M(N)) = N$ 4.8 Back-box instrumentation transparency complete 4.11 Back-box instrumentation transparency sound 5.1 Equivalence Between Deadlocks and Lock-on cycles 5.2 Deadlock Detection Algorithm p.14 6.1 Definition: Initial network and instrumentation p.16 6.2 Definition: Client p.16 6.3 Definition of well-formed SRPC client & server p.17 6.7 !Complete lock knowledge p.18 6.8 Alarm condition p.18 6.9 Alarm condition leads to alarm p.18 6.10 *Complete* lock knowledge leads to invariant p.19 6.11 Sound lock knowledge 6.12 Sound lock knowledge is an invariant 6.13 Deadlock Detection Preciseness 6.14 Eventual Deadlock Reporting

- Deadlockset
- Lemma 3.8 (Persistence of deadlocks).

Fragen

- Sollen wir paths noch zeigen?
- Locked Definition hinzufügen?
- Slide für “how to read LTS-semantics?”

Correct Black-Box Monitors for Distributed Deadlock Detection: Formalisation and Implementation

Disecting the Title

- We are given a network setting with communicating RPC Services
- **RPC**: Remote Procedure Call (Query, Response, Cast)
- **Deadlock**: Cyclic waiting for resources in such a network
- **Distributed**: No central authority (due to performance)
- **Blackbox**: No introspection of services, only communication behaviour
- **Monitors**: Wrappers around service that observe communication

Contributions

1. Establish criteria for correctness of deadlock detection
2. A formal model for RPC services and communication
3. A formal method to install distributed, black-box, outline monitors
4. A Monitoring algorithm that detects deadlock in a sound and complete way and proof its soundness
5. Implementation of DDMon (for Erlang/OTP)

Agenda

1. Formalising Processes & Services
2. Formalising Networks
3. Installing Monitors
4. Proof for Deadlock-detection
5. Deadlock Detection Algorithm
6. Proof of Soundness & Completeness

Wanted Properties

1. **Distributed**: No centralized component
2. **Blackbox and Outline**: Detect deadlocks by just observing the incoming and outgoing messages of the service they monitor
3. **Transparent**: Don't alter the execution
4. **Precise**: Detect Deadlock iff it existst

Nodes

Formalism: Services

- We define *Single-threaded Remote Procedure Calls* (SRPC)

Communication tag	$t := Q \mid R \mid C$	Name	$n := n_0 \mid n_1 \mid \dots$
Client	$n^\perp := n \mid \perp$	Process	$P := \dots$
Queue	$q := \varepsilon \mid n(t) \mid q \# q$	Service	$S := \langle q^i \mid P \mid q^o \rangle$

- The Behaviour of P must be compatible with an **abstract SRPC Process G**

Formalism: LTS Semantic of abstract SRPC Process G

- We define the possible states of an abstract Process G. The LTS Semantics are defined in Figure 1 using the transitions labels γ .

$$G := \text{Ready} \mid \text{Working}(n_c^\perp) \mid \text{Locked}(n_c^\perp, n_s)$$

$$\gamma := ?n(t) \mid !n(t) \mid \tau$$

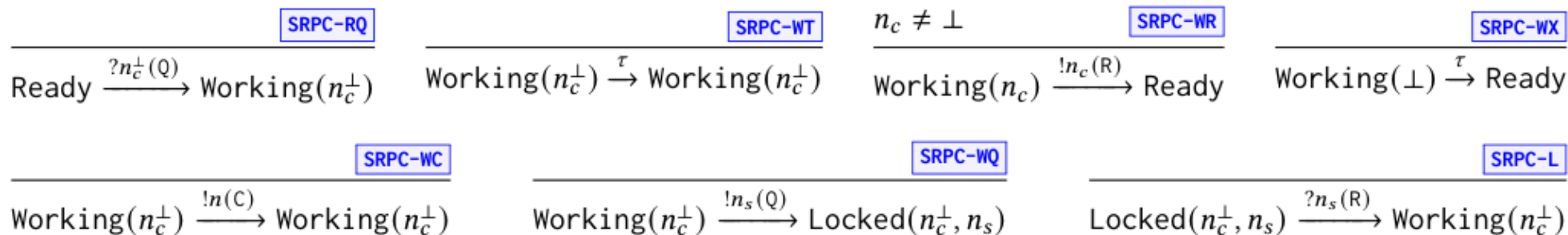


Figure 1: LTS semantics of the abstract SRPC process G

Formalism: LTS Semantic of Services

- We extend a process P with queues and search operations to form a service S . The lts-semantics in Figure 2 are using the transition labels in β . δ is a transition label for queues.

$$\delta := ![n(t)]$$

$$\beta := ?n(t) \mid !n(t) \mid \tau(?n(t)) \mid \tau(!n(t)) \mid \tau(\tau)$$

$$\begin{array}{c}
 \frac{}{n(t) ++ q \xrightarrow{![n(t)]} q} \text{QUE-FIND} \quad \frac{q_1^i = n(t) ++ q_0^i}{\langle q_0^i | P | q^o \rangle \xrightarrow{?n(t)} \langle q_1^i | P | q^o \rangle} \text{SER-I} \quad \frac{q_0^o = n(t) ++ q_1^o}{\langle q^i | P | q_0^o \rangle \xrightarrow{!n(t)} \langle q^i | P | q_1^o \rangle} \text{SER-O} \quad \frac{P_0 \xrightarrow{\tau} P_1}{\langle q^i | P_0 | q^o \rangle \xrightarrow{\tau(\tau)} \langle q^i | P_1 | q^o \rangle} \text{SER-TT} \\
 \\
 \frac{n(t) \neq n'(t') \quad q \xrightarrow{![n'(t')]} q'}{n(t) ++ q \xrightarrow{![n'(t')]} n(t) ++ q'} \text{QUE-SEEK} \quad \frac{q_0^i \xrightarrow{![n(t)]} q_1^i \quad P_0 \xrightarrow{?n(t)} P_1}{\langle q_0^i | P_0 | q^o \rangle \xrightarrow{\tau(?n(t))} \langle q_1^i | P_1 | q^o \rangle} \text{SER-TI} \quad \frac{P_0 \xrightarrow{!n(t)} P_1 \quad q_1^o = n(t) ++ q_0^o}{\langle q^i | P_0 | q_0^o \rangle \xrightarrow{\tau(!n(t))} \langle q^i | P_1 | q_1^o \rangle} \text{SER-TO}
 \end{array}$$

Figure 2: LTS semantics of services

Networks

Formalism: Network

$$\alpha := \tau(\gamma @ n) \mid n_0 - (t) \rightarrow n_1$$

$$\begin{array}{c}
 \frac{N(n) \xrightarrow{\tau(\gamma)} S \quad \boxed{\text{NET-TAU}}}{N \xrightarrow{\tau(\gamma @ n)} N[n \leftarrow S]}
 \end{array}
 \qquad
 \begin{array}{c}
 \frac{N(n_0) \xrightarrow{!n_1(C)} S_0 \quad N[n_0 \leftarrow S_0](n_1) \xrightarrow{?\perp(Q)} S_1}{N \xrightarrow{n_0 - (C) \rightarrow n_1} N[n_0 \leftarrow S_0][n_1 \leftarrow S_1]} \quad \boxed{\text{NET-CAST}}
 \end{array}$$

$$\begin{array}{c}
 \frac{t \neq C \quad N(n_0) \xrightarrow{!n_1(t)} S_0 \quad N[n_0 \leftarrow S_0](n_1) \xrightarrow{?n_0(t)} S_1}{N \xrightarrow{n_0 - (t) \rightarrow n_1} N[n_0 \leftarrow S_0][n_1 \leftarrow S_1]} \quad \boxed{\text{NET-COM}}
 \end{array}$$

Figure 3: LTS semantics of services

Monitors

Formalism: Monitors

- We define a monitored Service $\hat{S}$:

$$\hat{m} := ? n(t) \mid !n(t) \mid ? n(\hat{p}) \mid !n(\hat{p})$$

$$\hat{q} := \varepsilon \mid \hat{m} \mid \hat{q} \# \hat{q}$$

$$\hat{S} := \langle \hat{q} \mid P \mid S \rangle$$

- With Monitored service action:

$$\hat{\beta} := \beta \mid ? n(\hat{p}) \mid !n(\hat{p}) \mid \hat{\tau}(? n(t)) \mid \hat{\tau}(!n(t)) \mid \hat{\tau}(? n(\hat{p}))$$

- A monitored Service: $\langle \hat{q} \mid \hat{M} \mid S \rangle$
- Monitor algorithm function: $\hat{\mathcal{A}} : \hat{\mathcal{M}} \times \hat{m} \rightarrow \hat{\mathcal{M}} \times \hat{q}$

Slide

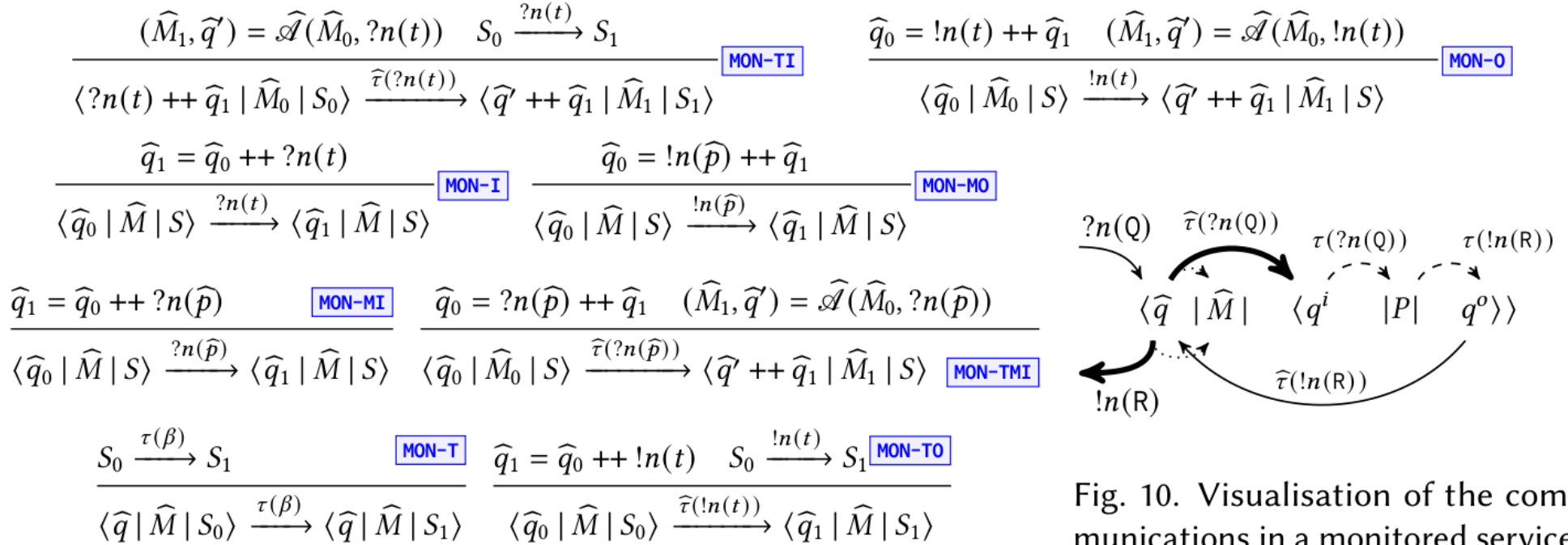


Fig. 9. LTS semantics of monitored services. In rule **MON-T**, β is either $?n(t)$, $!n(t)$, or τ (from Def. 3.4).

Fig. 10. Visualisation of the communications in a monitored service that immediately responds to an incoming query.

Figure 4: LTS semantics of monitored networks & visualization of communication.

Formalism: Monitored Network action

- The lts-semantics given in Figure 5 is using the transition labels $\hat{\alpha}$

$$\hat{\alpha} := \alpha \mid \hat{\tau}(\gamma @ n) \mid n_0 - (\hat{p}) \rightarrow n_1$$

$$\begin{array}{c}
\frac{\widehat{N}(n_0) \xrightarrow{!n_1(t)} \widehat{S}_0 \quad \widehat{N}[n_0 \leftarrow \widehat{S}_0](n_1) \xrightarrow{?n_0(t)} \widehat{S}_1 \quad t \neq \mathsf{C}}{N \xrightarrow{n_0-(t) \rightarrow n_1} N[n_0 \leftarrow S_0][n_1 \leftarrow S_1]} \quad \boxed{\text{MNET-SCOM}}
\end{array}
\quad
\frac{\widehat{N}(n_0) \xrightarrow{!n_1(\mathsf{C})} \widehat{S}_0 \quad \widehat{N}[n_0 \leftarrow \widehat{S}_0](n_1) \xrightarrow{?\perp(Q)} \widehat{S}_1}{N \xrightarrow{n_0-(t) \rightarrow n_1} N[n_0 \leftarrow S_0][n_1 \leftarrow S_1]} \quad \boxed{\text{MNET-SCAST}}$$

$$\frac{\widehat{N}(n_0) \xrightarrow{!n_1(\widehat{p})} \widehat{S}_0 \quad \widehat{N}[n_0 \leftarrow \widehat{S}_0](n_1) \xrightarrow{?n_0(\widehat{p})} \widehat{S}_1}{\widehat{N} \xrightarrow{n_0-(\widehat{p}) \rightarrow n_1} \widehat{N}[n_0 \leftarrow \widehat{S}_0][n_1 \leftarrow \widehat{S}_1]} \quad \boxed{\text{MNET-COM}}$$

$$\frac{\widehat{N}(n) \xrightarrow{\tau(\gamma)} \widehat{S}}{\widehat{N} \xrightarrow{\tau(\gamma @ n)} \widehat{N}[n \leftarrow \widehat{S}]} \quad \boxed{\text{MNET-STAU}}$$

$$\frac{\widehat{N}(n) \xrightarrow{\widehat{\tau}(\widehat{\beta})} \widehat{S}}{\widehat{N} \xrightarrow{\widehat{\tau}(\widehat{\beta} @ n)} \widehat{N}[n \leftarrow \widehat{S}]} \quad \boxed{\text{MNET-TAU}}$$

Figure 5: LTS semantics of monitored networks

Instrumentation

- We now discuss the process of transforming a network into a monitored network in which communication is observed by monitors
- Monitor instrumentation: $\mathcal{M} : \mathcal{N} \rightarrow \hat{\mathcal{N}}$
- Deinstrumentation: $\mathcal{M}^{-1} : \hat{\mathcal{N}} \rightarrow \mathcal{N}$
- This transformation is *transparent* (Theorem 4.11)

$$(1) \ N(n) = \mathbf{R} \text{ implies } (\mathcal{M}(N))(n) = \langle \hat{q}, \hat{M}, \mathbf{R} \rangle$$

$$(2) \ (\mathcal{M}(N))(n) = \langle \hat{q}, \hat{M}, \mathbf{R} \rangle \text{ implies } N(n) = \mathbf{R}$$

Deadlock Detection using Probes

- We use an edge-chasing inspired technique and probes
- And send probes as soon as we appear to be locked
- Monitor states $\widehat{M}$ is a record with the fields: { probe | waiting | alarm }

$$\begin{aligned}
\mathcal{A}(\widehat{M}, ?n^\perp(Q)) &= \begin{cases} (\widehat{M}, \epsilon) & \text{if } n^\perp = \perp & \text{ALG-CI} \\ (\widehat{M}[\text{waiting} \leftarrow \widehat{M}.\text{waiting} \cup \{n^\perp\}], \epsilon) & \text{if } \widehat{M}.\text{probe} = \perp & \text{ALG-QI-NL} \\ (\widehat{M}[\text{waiting} \leftarrow \widehat{M}.\text{waiting} \cup \{n^\perp\}], !n(\widehat{M}.\text{probe})) & \text{else} & \text{ALG-QI-L} \end{cases} \\
\mathcal{A}(\widehat{M}, !n(Q)) &= (\widehat{M}[\text{probe} \leftarrow \text{freshProbe()}], \epsilon) & \text{ALG-QO} \\
\mathcal{A}(\widehat{M}, ?n(R)) &= (\widehat{M}[\text{probe} \leftarrow \perp], \epsilon) & \text{ALG-RI} \\
\mathcal{A}(\widehat{M}, !n(R)) &= (\widehat{M}[\text{waiting} \leftarrow \widehat{M}.\text{waiting} \setminus \{n\}], \epsilon) & \text{ALG-RO} \\
\mathcal{A}(\widehat{M}, ?n(\widehat{p})) &= \begin{cases} (\widehat{M}[\text{alarm} \leftarrow \text{true}], \epsilon) & \text{if } \widehat{M}.\text{probe} = \widehat{p} & \text{ALG-P-ALARM} \\ (\widehat{M}, \epsilon) & \text{if } \widehat{M}.\text{probe} = \perp & \text{ALG-P-IGNORE} \\ (\widehat{M}, [!n'(\widehat{p}) \text{ for } n' \in \widehat{M}.\text{waiting}]) & \text{otherwise} & \text{ALG-P-PROPAGATE} \end{cases} \\
\mathcal{A}(\widehat{M}, !n(C)) &= (\widehat{M}, \epsilon) & \text{ALG-CO}
\end{aligned}$$

Figure 6: Implementation of $\mathcal{A}$

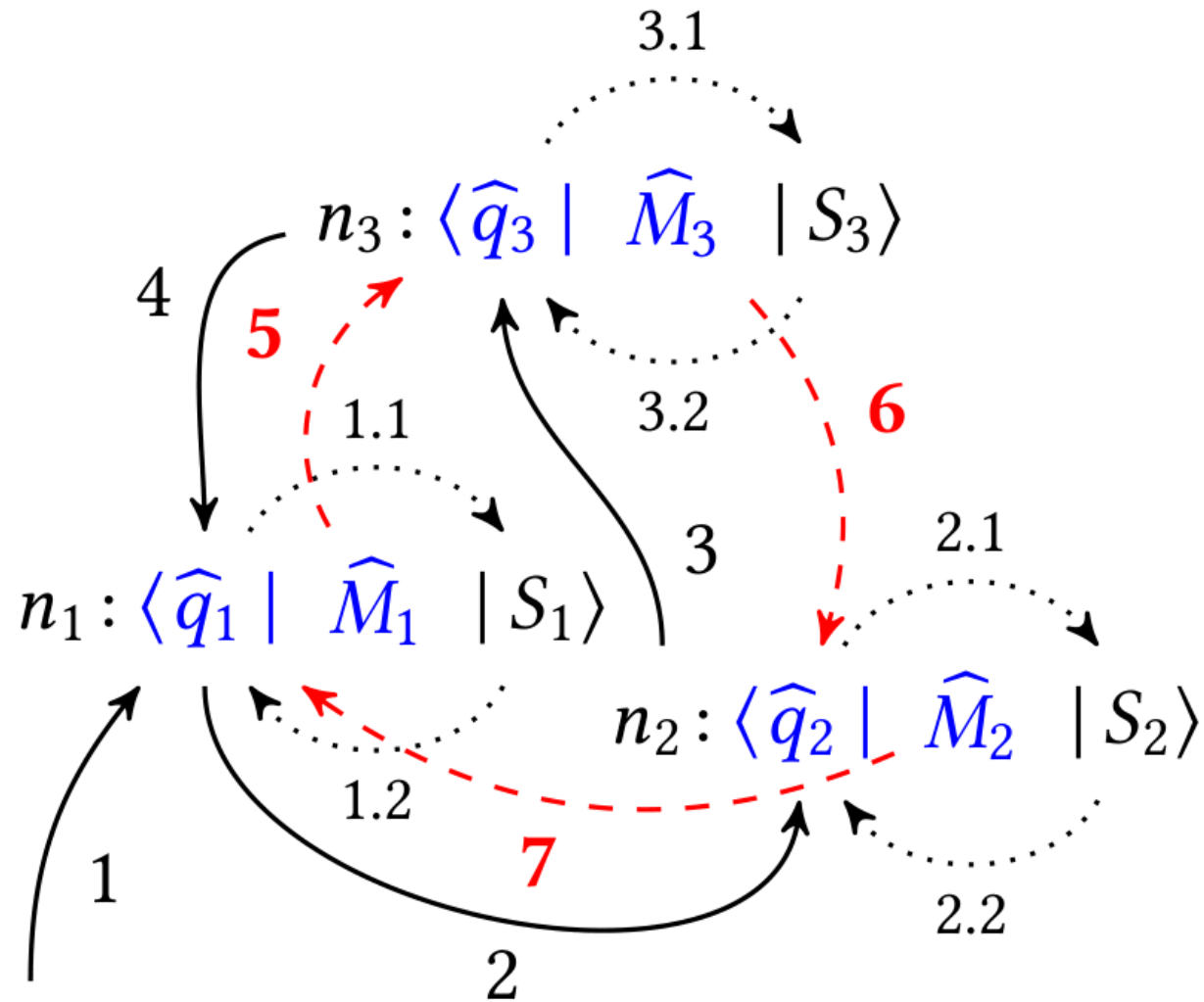


Figure 7: Deadlock detection algorithm example with three nodes.

Correctness Proofs

- We strive for an algorithm that is both *sound* and *complete*
- Preliminaries:
 - 6.1 Definition: Initial network and instrumentation
 - 6.2 Definition: *Client*
 - 6.3 Definition of well-formed SRPC *client & server*
 - 6.4 Definition of well-formed SRPC *network*
 - 6.5 Well-formedness is an invariant
- Outline of the completeness-proof:
 - We try to prove that if there is a lock, we see it in the network
 - 6.7 Complete lock knowledge p.18
 - 6.8 Alarm condition p.18
 - 6.9 Alarm condition leads to alarm p.18
 - 6.10 *Complete* lock knowledge leads to invariant p.19

Soundnessproof

- Outline of the soundness proof:
 - 6.11 Sound lock knowledge
 - 6.12 Sound lock knowledge is an invariant

THEOREM 6.13 (DEADLOCK DETECTION PRECISENESS). *Let N be an initial SRPC network, and $\mathcal{M}$ be the initial deadlock detection instrumentation for N . Then, for any monitored network $\widehat{N}'$ and monitored path $\widehat{\sigma}$ such that $\mathcal{M}(N) \xrightarrow{\widehat{\sigma}} \widehat{N}'$:*

Completeness: *If there is a deadlock in $\mathcal{M}^{-1}(\widehat{N}')$, then there are $\widehat{N}'', \widehat{\sigma}'$ such that $\widehat{N}' \xrightarrow{\widehat{\sigma}'} \widehat{N}''$ and $\widehat{N}''$ reports a deadlock.*

Soundness: *If $\widehat{N}'$ reports a deadlock, then there is a deadlock in $\mathcal{M}^{-1}(\widehat{N}')$.*

Figure 8: Theorem 6.13 (Deadlock Detection Preciseness)

Evaluation

- Overhead is negligible

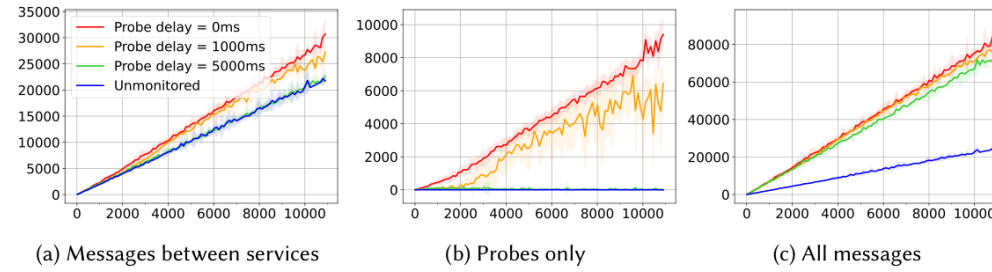


Fig. 15. Number of message exchanges (measured on the y -axis) vs. number of services in the network (x -axis).
(Averages of 20 runs for network sizes < 8500 , 10 runs for sizes $< 10k$, and 5 runs for sizes $\geq 10k$; standard deviation outlined in the background)

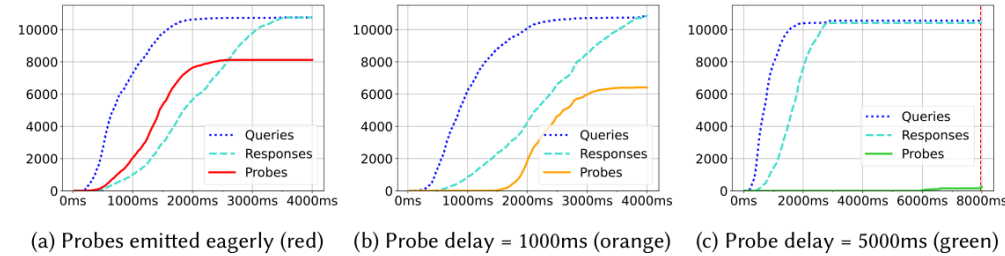


Fig. 16. Time series showing executions of monitored systems with 11 000 services. The execution in Fig. 16c ends with a deadlock and its detection is marked with a vertical red line.

Thank you for listening!